

1. prank_message.py — repeated popup

Create:

- prank_message.py

Use this version:

```
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()
root.withdraw()

while True:
    answer = messagebox.askokcancel(
        "Prank Message",
        "😄 You found the prank! Click OK for another message.\n\n"
        "Click Cancel to stop."
    )

    if not answer:
        break

root.destroy()
```

Run

- python prank_message.py

2. JavaScript — open tabs with a 5-tab limit

```
let tabsOpened = 0;
const maxTabs = 5;

const tabTimer = setInterval(() => {
    if (tabsOpened >= maxTabs) {
```

```
clearInterval(tabTimer);  
console.log("Stopped: maximum of 5 tabs reached.");  
return;  
}
```

```
window.open("https://example.com", "_blank");  
tabsOpened++;
```

```
console.log(`Opened tab ${tabsOpened}/${maxTabs}`);  
}, 2000);
```

It attempts to open:

- Tab 1 → after 2 seconds
- Tab 2 → after 4 seconds
- Tab 3 → after 6 seconds
- Tab 4 → after 8 seconds
- Tab 5 → after 10 seconds

3. file_spreader.py — safe worm-like simulation

Create:

- payload.txt

with:

- This is a harmless cybersecurity worm simulation.
- No malicious activity is performed.

Now create:

- file_spreader.py

Use:

```
from pathlib import Path  
import shutil
```

```

# CHANGE THIS to a dedicated lab directory
TARGET_DIR = Path("worm_lab")

payload = Path("payload.txt")

if not TARGET_DIR.exists():
    print(f"Directory not found: {TARGET_DIR}")
    exit()

for folder in TARGET_DIR.iterdir():

    if folder.is_dir():
        destination = folder / payload.name

        shutil.copy(payload, destination)

        print(f"Copied payload to: {destination}")

```

Prepare your lab

Create:

```

worm_lab/
├── folder1/
├── folder2/
├── folder3/
└── folder4/

```

Keep file_spreader.py and payload.txt outside worm_lab:

```

Malware_Lab/
├── file_spreader.py
├── payload.txt
└── worm_lab/
    ├── folder1/
    ├── folder2/
    ├── folder3/
    └── folder4/

```

Run:

- python file_spreader.py

4. Add worm_log.txt to the popup program

Now modify prank_message.py so every popup is recorded.

```
import tkinter as tk
from tkinter import messagebox
from datetime import datetime

root = tk.Tk()
root.withdraw()

log_file = "worm_log.txt"

while True:

    current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open(log_file, "a", encoding="utf-8") as log:
        log.write(f"Popup displayed: {current_time}\n")

    answer = messagebox.askokcancel(
        "Prank Message",
        "😁 You found the prank!\n\n"
        "Click OK for another message.\n"
        "Click Cancel to stop."
    )

    if not answer:
        break

root.destroy()
```

```
print("Prank stopped.")
```

After running it several times, worm_log.txt might contain:

- Popup displayed: 2026-09-17 11:30:04
- Popup displayed: 2026-09-17 11:30:07
- Popup displayed: 2026-09-17 11:30:11
- Popup displayed: 2026-09-17 11:31:25

Why "a" is used

- `open(log_file, "a")`

5. Three ways antivirus software can detect a worm

1. Signature-based detection

- Antivirus software can compare files against known malware signatures.
- For example, if a particular worm has a known byte pattern or hash associated with it, antivirus software can recognize the file and flag it.
- Limitation: This is less effective against completely new or heavily modified malware.

2. Behavior-based detection

- Antivirus software can monitor what a program actually does.
- A program that suddenly:
 - creates many copies of itself,
 - modifies many files,
 - attempts network connections,
 - or repeatedly launches processesmay trigger behavioral detection.
- This is particularly relevant to worms because self-propagation is a characteristic behavior.

3. Network/activity monitoring

- Security software can monitor unusual network activity.

- For example, a computer attempting to contact many other machines or repeatedly attempting connections to vulnerable services could indicate worm-like propagation.
- Modern endpoint security can combine network behavior with process and file activity to identify suspicious patterns.